

Hierarchical Locomotion Control for Quadruped Oval-Track Racing

EEC 289A- Final Project Track 2

Vandana Mansur & Pranav Manimaran

Spring 2026

1. Introduction

This report describes our approach to the Go2 oval-track racing challenge, where a simulated Unitree Go2 quadruped must complete a 200 m oval track in MuJoCo as quickly and stably as possible. The system uses a hierarchical control architecture: a learned high-level planner maps compact track observations to velocity commands, which a low-level reinforcement learning (RL) policy converts into 12 joint-position targets at 50 Hz.

Our final system completes the full 200 m lap in 107.64 seconds at 1.86 m/s average speed with zero falls, zero boundary violations, and a composite score of 0.973.

2. System Architecture

The controller follows a two-level hierarchy with clearly separated concerns:

Track Observation (5D) → High-Level Planner → [vx, vy, yaw_rate] → Low-Level PPO Policy → 12 Joint Actions

2.1 Observation Design

The high-level planner receives a compact 5-dimensional track observation computed from the robot's base position and the official oval centerline:

Index	Field	Description
0	lap_fraction	Arc-length progress s/L, normalized to [0, 1]
1	lateral_error_norm	Signed cross-track error / half_width
2	boundary_margin_norm	Distance to nearest boundary / half_width
3	heading_error_rad	Angle between robot heading and track tangent (rad)
4	curvature_norm	Local curvature $\times$ turn_radius (0 on straights, ± 1 on turns)

The low-level policy observes a 48-dimensional proprioceptive state vector including joint positions, joint velocities, IMU orientation, angular velocity, and the 3D velocity command. It does not receive any track information directly; all navigation is handled by the high-level planner.

2.2 Action Design

The high-level planner outputs a 3D velocity command $[v_x, v_y, \text{yaw_rate}]$ in the robot’s local frame. The low-level policy maps the 48D state observation to 12 joint-position targets (3 joints $\times$ 4 legs), scaled by an `action_scale` of 0.5 and applied through PD actuators at each control step ($\text{dt} = 0.02$ s, i.e., 50 Hz). The simulation substep is 0.004 s (250 Hz).

2.3 Reward Design

The low-level PPO policy is trained with MuJoCo Playground’s Go2 joystick reward, which combines velocity-tracking accuracy with regularization terms:

- **Primary:** Velocity tracking: rewards minimizing the error between commanded and achieved $[v_x, v_y, \text{yaw_rate}]$.
- **Regularization:** Action-rate penalty (weight -0.015): penalizes rapid changes in joint targets to encourage smooth gaits.
- **Efficiency:** Energy penalty (weight -0.0008): penalizes $|\text{torque} \times \text{velocity}|$ to promote efficient locomotion.

The reward weights were tuned through the two-stage curriculum described in Section 3. Stage 1 uses slightly stronger regularization (`action_rate` = -0.02 , `energy` = -0.0015) to establish stable gaits before the high-speed stage relaxes them.

3. Low-Level Command Tracking: Two-Stage Curriculum

A critical design decision was retraining the low-level policy with a two-stage curriculum that progressively expands the command space. The default configuration only trains forward velocity (`vy` = 0, `yaw` = 0), which is insufficient for track racing where the robot must navigate turns.

3.1 Stage 1: Forward Locomotion (10M steps)

The first stage trains basic forward locomotion with a restricted command range:

- $v_x \in [0.0, 1.2]$ m/s, `vy` = 0, `yaw_rate` = 0
- `command_keep_prob` = [1.0, 0.0, 0.0] — only forward commands sampled

This produces a stable walking gait at moderate speeds. Stage 1 serves as a pre-training phase that provides a good initialization for Stage 2.

3.2 Stage 2: High-Speed Turning (18M steps)

The key modification: we expanded Stage 2 to the `student_stage2_goal` command ranges, which include lateral velocity and yaw rate:

- $v_x \in [-0.6, 2.5]$ m/s, $v_y \in [-0.3, 0.3]$ m/s, $\text{yaw} \in [-1.0, 1.0]$ rad/s
- `command_keep_prob` = [1.0, 0.4, 0.6] — `vy` sampled 40%, `yaw` 60% of the time

This change was essential. Without it, the policy cannot track the planner’s turning commands, resulting in near-zero track progress (we observed only 0.001 m in 5 seconds with the original forward-only policy). Stage 2 fine-tunes from the Stage 1 checkpoint, preserving the stable gait while adding turning capability.

3.3 Network Architecture

Both stages use identical network architectures:

- Policy network: $48 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 12$ (SiLU activation, tanh-normal distribution)
- Value network: $123 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 1$ (privileged state input)
- PPO with 1024 parallel environments, batch size 256, observation normalization

Total training budget: 28M environment steps (within the 30M leaderboard cap), trained on a single Colab T4 GPU in approximately 6 hours.

4. Learned High-Level Planner

4.1 Planner Architecture: race_ff

We use the race_ff (race feedforward) planner, which combines feedforward speed profiling with Stanley-style lateral PD correction. Given the 5D track observation, it computes:

Forward velocity vx:

$$vx = speed_mps \times (1 - turn_speed_drop \times |curvature_norm|) \times margin^{margin_power} \times (1 - 0.35 \times |heading_error| / \pi)$$

$$vx = clip(vx, min_speed_mps, speed_mps)$$

The three multiplicative factors reduce speed based on: (1) track curvature (slow down in turns), (2) proximity to boundaries (slow down near edges), and (3) heading misalignment (slow down when facing the wrong direction).

Lateral velocity vy:

$$vy = clip(-k_lateral \times lateral_error_m, \pm max_lateral_speed_mps)$$

Yaw rate:

$$yaw_rate = clip(curvature_feedforward \times \kappa \times vx + k_heading \times heading_error, \pm max_yaw_rate_radps)$$

where $\kappa = curvature_norm / turn_radius$ is the physical curvature. The first term provides feedforward yaw anticipation for turns; the second is proportional heading correction.

4.2 Evolutionary Parameter Search

The 10 scalar parameters of race_ff were optimized using the evolutionary search in train_highlevel_mlp.py:

- Population size: 16 candidates per iteration
- Iterations: 12 (192 total evaluations)
- Evaluation: 90-second no-render rollout per candidate
- Fitness: composite of lap completion, distance, speed, and stability
- Mutation: Gaussian perturbation of parent parameters with adaptive step size

The search found lap-completing candidates (fitness > 1000) within the first iteration (candidate 4 at iter=0). Subsequent iterations refined the parameters for better speed and line-keeping. The final optimized parameters:

Parameter	Default	Optimized	Effect
speed_mps	0.45	2.50	Max straight-line speed
min_speed_mps	0.12	1.40	Minimum speed in turns
turn_speed_drop	0.35	0.10	Speed reduction factor on curves
k_heading	0.55	1.20	Heading correction gain
k_lateral	0.08	0.22	Lateral correction gain
curvature_feedforward	1.05	1.15	Turn anticipation gain
margin_power	0.65	0.40	Boundary proximity penalty exponent
stand_seconds	1.0	0.0	Initial standing delay

5. Results and Metrics

The final system was evaluated on the official 200 m oval track (turn radius 18.25 m, half-width 2.0 m) with a 120-second simulation budget.

5.1 Final Performance Metrics

Metric	Value	Score Component	Score
Lap completion	1.0 (100%)	Completion (45%)	1.000
Finish time	107.64 s	—	—
Valid distance	200.0 m	—	—
Mean progress speed	1.858 m/s	Speed (20%)	1.000
Fall	No	Stability (10%)	1.000
Boundary violation	No	Stability (10%)	1.000
RMS lateral error	0.203 m	Line keeping (20%)	1.000
Max lateral error	0.257 m	Line keeping (20%)	1.000
Energy proxy	16.95	Efficiency (5%)	0.455
Foot slip proxy	1.927	Efficiency (5%)	0.455
Composite Score			0.973

5.2 Scoring Formula

The composite score is computed as:

$$S = 0.45 \times \text{completion} + 0.20 \times \text{speed} + 0.20 \times \text{line_keeping} + 0.10 \times \text{stability} + 0.05 \times \text{efficiency}$$

Our system achieves maximum scores (1.0) on completion, speed, line-keeping, and stability. The only non-perfect component is efficiency (0.455), driven primarily by foot slip. The composite of 0.973 is near the theoretical ceiling of 1.0.

5.3 Progression Through Development

Run	Config Change	Distance	Speed	Lat. Error	Composite
Baseline	Starter PD, forward-only policy	0.001 m	0.0002 m/s	0.011	0.350
Run 2	race_ff + vy/yaw training, 90s	171.0 m	1.90 m/s	0.22	0.907
Run 3	Tuned race_ff params, 90s	174.3 m	1.94 m/s	0.17	0.914
Run 4	Aggressive params (too fast)	193.7 m	1.76 m/s	0.42	0.945
Final	Balanced params + evo search, 120s	200.0 m	1.86 m/s	0.20	0.973

The progression shows how each design decision contributed: enabling vy/yaw training was the single largest improvement (0.35 $\rightarrow$ 0.91), followed by the evolutionary parameter search (0.91 $\rightarrow$ 0.97).

6. Sim-to-Real Transfer Considerations

Several aspects of our design would facilitate or hinder deployment on a physical Go2 robot:

Design choices that help transfer:

- **Proprioceptive state:** The hierarchical separation means the low-level policy only needs proprioceptive observations (joint encoders, IMU), which are directly available on the real robot. No vision or lidar is required.
- **Smooth gaits:** The action-rate and energy penalties in the reward encourage smooth, efficient gaits that are less likely to damage actuators or exceed torque limits.

- **Modular design:** The two-stage curriculum produces a general-purpose velocity-tracking policy, not a track-specific one. The same low-level policy could be reused with different high-level planners.

Challenges for transfer:

- **Foot slip:** The high foot-slip proxy (1.93) suggests the simulated gaits rely on ground contact conditions that may differ on real surfaces. Domain randomization of friction coefficients would help.
- **Terrain variety:** Our policy was trained on flat terrain only. Real-world tracks have surface irregularities, banking, and slope variations.
- **Actuator dynamics:** Real actuators have delays, backlash, and compliance that are not fully captured in MuJoCo's PD model. System identification or learned residual adaptation would be needed.
- **Localization:** On a real track, the 5D observation would require a localization system (e.g., motion capture or visual odometry) to compute lap fraction and lateral error.

7. Failed Ideas and Lessons Learned

Failed Idea 1: Aggressive turn parameters for sub-90s lap time

We attempted to achieve a sub-90s lap by setting `turn_speed_drop` to 0.05 and `min_speed_mps` to 1.60, forcing the robot to maintain near-maximum speed through turns. This backfired: the robot could not track the aggressive commands smoothly, resulting in lateral oscillations (RMS lateral error jumped from 0.17 to 0.42 m) that actually reduced the average forward speed from 1.94 to 1.76 m/s. The lesson: commanding higher speeds does not guarantee faster lap times when the low-level policy cannot track them stably. A smoother, slightly slower command profile produces better actual speed through reduced corrective motion.

Failed Idea 2: Training with forward-only commands

Our initial training used the default Stage 2 command ranges (`vx` only, `vy` = 0, `yaw` = 0). The resulting policy could walk forward but could not turn, producing only 0.001 m of track progress in 5 seconds. We discovered that the `student_stage2_goal` ranges (including `vy` $\in [-0.3, 0.3]$ and `yaw` $\in [-1.0, 1.0]$) must be explicitly applied to the training configuration. This single fix was the most impactful change in the entire project, improving the composite score from 0.35 to 0.91.

Failed Idea 3: Running eval and video rendering in a single cell

The original notebook ran the simulation rollout and video rendering in one cell. Video rendering in MuJoCo (with `MUJOCO_GL=egl` on Colab) is very slow, and the cell was repeatedly killed by Ctrl+C or Colab timeouts before `results.json` was written (it is saved after the video). We fixed this by splitting into a fast no-render eval cell (writes `results.json` immediately) and a separate optional video cell. Engineering lesson: always save critical outputs before expensive optional steps.

8. Individual Contributions

Task	Vandana	Pranav
Low-level PPO training & curriculum design	Lead	Support
Stage 2 command range fix (vy/yaw)	Lead	Review
High-level planner architecture (race_ff)	Collaborate	Collaborate
Evolutionary parameter search setup	Support	Lead
Planner parameter tuning & analysis	Collaborate	Collaborate
Notebook pipeline (eval, bundle, configs)	Lead	Support
Debugging ^C / video rendering issues	Lead	Support
Final evaluation & metrics analysis	Support	Lead
Report writing	Collaborate	Collaborate

9. References

- [1] Schulman, J. et al. “Proximal Policy Optimization Algorithms.” arXiv:1707.06347, 2017.
- [2] Google DeepMind. MuJoCo Playground. github.com/google-deepmind/mujoco_playground, 2024.
- [3] Todorov, E., Erez, T., and Tassa, Y. “MuJoCo: A physics engine for model-based control.” IROS, 2012.
- [4] Stanley, G. M. “The Stanley controller for autonomous driving.” Journal of Field Robotics, 2006.
- [5] Hansen, N. “The CMA Evolution Strategy: A Tutorial.” arXiv:1604.00772, 2016.